

Return-Abuse Sentinel — Project Memory and Handover Notes

1. Project at a glance

Project name: Return-Abuse Sentinel

Purpose: A defense-only AI risk-management platform for merchants. It detects potentially abusive return/refund requests, routes them by risk level, produces an explainable evidence pack, and leaves the final decision to a human reviewer.

Buildathon context: Designed for the Razorpay AI Risk Manager theme:

Stop merchants losing money to fraud, returns, and chargebacks. Build a working detector, verifier, or auto-responder for one class of loss, with measured precision/recall on a held-out set and an honest false-positive-cost model. The project must be defense-only.

Loss class selected: Return/refund abuse.

Examples of return abuse the system is intended to help identify:

- Serial returners with unusually high return rates
- Refund-to-keep abuse
- Bracketing: ordering multiple sizes/variants and returning most of them
- Suspicious or templated return-claim text
- AI-generated or suspicious damage-photo signals
- Empty-box or item-swap indicators from parcel-weight mismatches
- Coordinated abuse rings using shared device, address, or payment fingerprints
- Incomplete carrier scan trails

The system is **defense-only**. It does not automatically deny a return, reverse a refund, block an account, contact a customer, or perform any irreversible action. It outputs a risk score, a routing recommendation, and evidence for a human reviewer.

2. The business problem solved

Returns and refund abuse can quietly destroy a merchant's margins even when payment fraud is low. A merchant may lose money through the product itself, shipping, warehouse handling, refunds, chargebacks, and customer-support work.

A simple rule like "block everyone with more than three returns" is unfair and costly. It creates **false positives**: genuine customers are delayed or treated as suspicious, which can harm trust

and customer satisfaction.

Return-Abuse Sentinel solves this by combining several risk signals into a calibrated probability score, then routing cases into three operational bands:

Band	Meaning	System action
Green	Low return-abuse risk	Auto-approve the return/refund flow according to merchant policy, e.g. refund after carrier-scan confirmation
Yellow	Uncertain/moderate risk	Hold for low-cost warehouse or photo inspection before refunding; optionally offer exchange/store credit
Red	High risk	Do not auto-refund or auto-deny. Create an evidence pack and route to a human fraud-review queue

The key idea is that a merchant should not use a single arbitrary 0.5 ML threshold. The appropriate decision boundary depends on merchant economics:

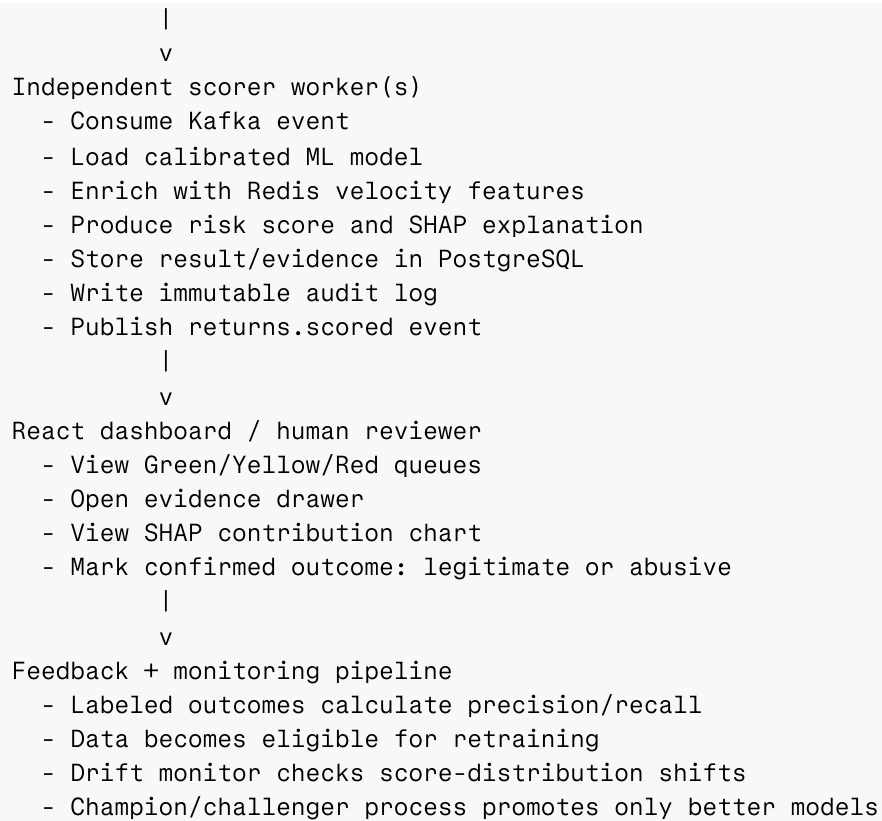
- How expensive is a manual review?
- How expensive is warehouse inspection?
- How costly is customer friction caused by delaying a legitimate refund?
- How much money is lost when abusive returns are approved?

The project includes a per-merchant threshold optimizer that minimizes **expected ₹ loss** under that merchant's own cost profile.

3. System workflow

End-to-end workflow

```
Merchant/OMS/Frontend request
|
v
FastAPI API gateway
- Validates payload using Pydantic
- Authenticates JWT/API key user
- Enforces merchant-level access control
- Deduplicates repeat order submissions through Redis
|
v
Rule-based pre-screen
- High-risk conditions use synchronous path
- Normal cases use asynchronous path
|
+--> Synchronous path
|   - API scores immediately
|   - Returns score/band/evidence in response
|
+--> Asynchronous path
    - API saves pending record in PostgreSQL
    - API publishes event to Kafka topic: returns.incoming
```



Example: suspicious return

A customer has a 65% historical return rate, two refund-to-keep incidents, high bracketing behavior, suspicious claim text, a high AI-artifact score for the damage image, and a parcel-weight mismatch.

1. Merchant submits the return through POST /returns/score.
2. FastAPI validates every field and checks that the authenticated user belongs to merchant M001.
3. Because the request does not hit a synchronous pre-screen condition, FastAPI responds rapidly with:

```
{
  "order_id": "ORD900001",
  "mode": "async",
  "status": "pending"
}
```

4. Kafka sends the event to a scorer worker.
5. The scorer loads the LightGBM model, applies isotonic probability calibration, calculates SHAP feature contributions, and persists the result.
6. In the live deployment, the scorer returned:

```
Scored ORD900001: band=Red score=92.8
```

7. PostgreSQL confirmed the record was persistently saved:

```
ORD900001 | M001 | scored | Red | 0.928
```

8. The dashboard can show the case in the Red queue and present the reason/evidence pack to a human reviewer.

4. Architecture and technology

Backend services

Component	Technology	Responsibility
API gateway	FastAPI + Uvicorn	REST API, Swagger docs, input validation, authorization, sync/async routing
Data validation	Pydantic v2	Strict validation of every incoming return field
Database	PostgreSQL 16	Persistent return records, merchant config, audit logs, model registry, daily metrics
ORM	SQLAlchemy	Python-to-PostgreSQL database models
Schema migration	Alembic	Versioned database schema creation/migration
Queue	Kafka + Zookeeper	Asynchronous event delivery between API and scorer workers
Feature cache	Redis	Request deduplication and rolling velocity features
Scoring worker	Python + aiokafka	Kafka consumer, model scoring, evidence persistence
ML model	LightGBM	Main return-abuse classification model
Calibration	Isotonic calibration	Makes output probabilities more meaningful for cost-based decisions
Explainability	SHAP TreeExplainer	Per-prediction feature contributions/waterfall explanation
Monitoring	Prometheus + Grafana	Metrics collection and dashboarding
Frontend	React + TypeScript + Vite	Operations dashboard with live queues and evidence drawer
Frontend data	TanStack Query + Axios	API calls, polling/cache, authentication header injection
Charts	Recharts	SHAP contribution chart and threshold tradeoff chart
Packaging/deployment	Docker Compose	One-command local stack startup

Docker Compose services

The local stack starts these services:

- postgres: PostgreSQL database
- redis: in-memory feature cache/dedup store

- `zookeeper`: Kafka coordination service
- `kafka`: event queue
- `api`: FastAPI API gateway
- `scorer`: two scorer worker replicas
- `webhook_dispatcher`: intended to send downstream merchant webhook events
- `frontend`: React dashboard exposed at `http://localhost:3000`
- `prometheus`: metrics exposed at `http://localhost:9090`
- `grafana`: observability UI exposed at `http://localhost:3001`

Important local URLs

URL	Purpose
<code>http://localhost:3000</code>	React operations dashboard
<code>http://localhost:8000/docs</code>	FastAPI interactive Swagger documentation
<code>http://localhost:8000/health</code>	API/model health check
<code>http://localhost:9090</code>	Prometheus
<code>http://localhost:3001</code>	Grafana

5. Key ML design

Input features

The model uses the following return-level features:

- `account_age_days`
- `past_orders`
- `return_rate_hist`
- `refund_to_keep_count`
- `bracketing_score`
- `claim_text_susp`
- `image_ai_artifact_score`
- `weight_mismatch_kg`
- `shared_identifier_flag`
- `carrier_scan_gap`
- `days_to_claim`
- `order_value`
- `category`

Why calibrated probabilities matter

Raw classifier scores are not always probabilities. A score of 0.8 might not actually mean an 80% abuse rate. Isotonic calibration learns a mapping from raw model outputs to observed outcomes using a held-out calibration dataset.

In the sandbox reference implementation:

- Uncalibrated AUC: approximately 0.8073
- Calibrated AUC: approximately 0.8061
- Uncalibrated Brier score: approximately 0.0908
- Calibrated Brier score: approximately 0.0902

The very similar AUC and slightly improved Brier score are expected: calibration should make probabilities more reliable without significantly changing model ranking.

Cost-aware thresholding

The project does not maximize only accuracy. It considers costs such as:

- Manual full-review cost: ₹150
- Yellow-band inspection cost: ₹40
- Customer-friction cost for a genuine Yellow case: ₹5
- Estimated loss for missed abuse: 60% of the order value

The optimizer searches combinations of Green/Yellow and Yellow/Red thresholds. It minimizes expected loss while requiring Green automation to cover a minimum share of volume (default: 55%). This prevents the model from sending nearly every case to human review just to look highly accurate.

The sandbox validation showed that different merchants get different threshold recommendations. For example:

```
Merchant A: green threshold around 0.096
Merchant B: green threshold around 0.121
```

This demonstrates per-merchant P&L optimization instead of using one global threshold.

Explainability

For each scored case, SHAP identifies which signals pushed risk up or down. The system converts those numerical contributions into plain language such as:

- “Historical return rate (65%) contributed +X percentage points to the risk score.”
- “Two prior refund-but-keep incidents contributed +Y percentage points.”
- “AI-generated damage photo signal contributed +Z percentage points.”
- “Parcel weight mismatch of 0.30 kg contributed +W percentage points.”

The dashboard visualizes top feature contributions in a horizontal bar chart called a SHAP waterfall-style chart.

6. Security and governance

Authentication

1. A merchant user is issued a raw API key once.
2. Only the SHA-256 hash of that API key is stored in PostgreSQL.
3. The client exchanges the raw API key through `POST /auth/token`.
4. The backend returns a short-lived signed JWT.
5. Protected endpoints require an HTTP Authorization header:

```
Authorization: Bearer <JWT>
```

Roles

Role	Permissions
ops	View queues/evidence packs, score returns, record outcomes
admin	Ops permissions plus issue API keys, change thresholds, change merchant cost profile, manage configuration
auditor	Read-only access to audit and decision history; no configuration mutation

Audit trail

The system uses an append-only `audit_log` table to store:

- Who or what scored a return
- Which action occurred
- Entity type and ID
- Before/after configuration values where relevant
- Timestamp

Safety guarantee

A Red score is a human-review recommendation, never an automatic punishment. The system must not auto-block customers, reverse payments, auto-deny claims, or generate offense-capable fraud behavior.

7. Monitoring, drift, and continuous learning

Drift monitoring

Population Stability Index (PSI) checks whether the current score distribution is materially different from a baseline distribution.

- PSI below 0.2: considered stable in this prototype
- PSI above 0.2: alert for feature drift or a genuine new abuse pattern

Sandbox validation:

```
PSI with no distribution change: 0.0000  
PSI after simulated fraud-pattern shift: 0.4265
```

The simulated shift correctly exceeded the 0.2 alert threshold.

Outcome labeling

A human reviewer can mark a return as:

- 0: legitimate
- 1: confirmed abuse

Labels are stored on the return record. Only cases with known outcomes should be used for honest precision/recall reporting and retraining.

Retraining

`retrain.py` implements a champion/challenger model process:

1. Pull sufficiently many labeled cases from PostgreSQL.
2. Train a challenger model.
3. Evaluate challenger versus current champion on AUC and Brier score.
4. Promote only when AUC improves and probability calibration does not worsen beyond a small tolerance.
5. Recalculate merchant-specific thresholds after promotion.

This avoids blindly overwriting a production model with a worse new model.

8. Files and folders

Project root used during deployment:

```
C:\Projects\return-abuse-sentinel
```

Expected organized structure:


```
return-abuse-sentinel/
├── app/
│   ├── __init__.py
│   ├── config.py
│   ├── database.py
│   ├── api/
│   │   ├── __init__.py
│   │   ├── auth.py
│   │   ├── main.py
│   │   └── schemas.py
│   ├── models/
│   │   ├── __init__.py
│   │   └── db_models.py
│   ├── monitoring/
│   │   ├── __init__.py
│   │   └── drift_monitor.py
│   ├── pipelines/
│   │   ├── __init__.py
│   │   ├── ingest.py
│   │   ├── retrain.py
│   │   ├── threshold_optimizer.py
│   │   └── train.py
│   └── services/
│       ├── __init__.py
│       ├── feature_store.py
│       ├── kafka_client.py
│       ├── ml_core.py
│       ├── pdf_export.py
│       ├── scorer_worker.py
│       └── webhook_dispatcher.py
├── artifacts/
│   └── model_current.pkl # Created by bootstrap/training
├── frontend/
│   ├── Dockerfile
│   ├── index.html
│   ├── package.json
│   ├── tsconfig.json
│   ├── vite.config.ts
│   └── src/
│       ├── App.css
│       ├── App.tsx
│       ├── main.tsx
│       ├── vite-env.d.ts
│       └── api/client.ts
├── migrations/
│   ├── env.py
│   └── versions/001_initial_schema.py
├── monitoring/prometheus.yml
├── scripts/
│   ├── __init__.py
│   └── bootstrap.py
├── .env
├── .env.example
├── Dockerfile.api
├── Dockerfile.scorer
└── Dockerfile.webhook
```

```
|— docker-compose.yml
|— requirements.txt
|— alembic.ini
|— setup_structure.py
|— DEPLOY.md
```

9. Deployment history

Initial local deployment commands

```
cd C:\Projects\return-abuse-sentinel
python cleanup_folder.py
python check_files.py
python setup_structure.py
copy .env.example .env
```

A real JWT secret was generated and put in `.env`:

```
python -c "import secrets; print(secrets.token_hex(32))"
```

Then Docker Compose was started:

```
docker compose up -d --build
```

The API, database, queue, workers, and frontend were subsequently verified through:

```
docker compose ps
docker compose logs --since=2m scorer
```

API health was confirmed from PowerShell:

```
Invoke-RestMethod http://localhost:8000/health
```

Successful output:

```
status model_loaded
-----
ok      True
```

First live scoring result

A suspicious request named ORD900001 was submitted with POST `/returns/score`.

The API correctly responded asynchronously:

```
{
  "order_id": "ORD900001",
  "mode": "async",
  "status": "pending",
  "risk_score": null,
  "band": null,
  "recommended_action": null,
  "evidence": null
}
```

This was correct behavior: API response was intentionally fast, while the Kafka worker performed scoring separately.

The worker then logged:

```
Scored ORD900001: band=Red score=92.8
```

PostgreSQL verified persistence:

```
docker compose exec postgres psql -U sentinel -d sentinel_db -c "SELECT order_id, me
```

Output:

```
ORD900001 | M001 | scored | Red | 0.9279999999999999
```

10. Errors encountered and corrections made

This section documents every important deployment/debugging issue so future maintainers do not repeat the same confusion.

Error 1: Docker daemon unavailable

Observed error:

```
failed to connect to the docker API at npipe:////./pipe/docker_engine
```

Cause: Docker Desktop engine was not fully running.

Fix: Start Docker Desktop and wait until the Docker engine is ready. Verify with:

```
docker ps
```

Error 2: Obsolete Compose version warning

Observed warning:

```
the attribute `version` is obsolete, it will be ignored
```

Cause: Modern Docker Compose v2 does not need `version: "3.9"`.

Fix: Remove the first line from `docker-compose.yml`:

```
version: "3.9"
```

This was only a warning, not a failure.

Error 3: React frontend TypeScript build failure

Observed error:

```
Property 'env' does not exist on type 'ImportMeta'.
```

Files involved:

```
frontend/src/App.tsx  
frontend/src/api/client.ts
```

Cause: TypeScript did not know Vite's `import.meta.env` typings.

Fix: Create:

```
frontend/src/vite-env.d.ts
```

with exactly:

```
/// <reference types="vite/client" />
```

Then rebuild frontend:

```
docker compose build frontend --progress=plain  
docker compose up -d --build frontend
```

Error 4: sklearn.frozen module missing

Observed error:

```
ModuleNotFoundError: No module named 'sklearn.frozen'
```

This caused both API and scorer containers to restart because both import `app/services/ml_core.py`.

Cause: The first version of `ml_core.py` used:

```
from sklearn.frozen import FrozenEstimator
```

but that module was unavailable in the installed scikit-learn version.

Fix: In `app/services/ml_core.py`:

1. Delete:

```
from sklearn.frozen import FrozenEstimator
```

2. Replace:

```
self.calibrated = CalibratedClassifierCV(FrozenEstimator(self.pipeline), method="iso")
self.calibrated.fit(X_calib, y_calib)
```

with:

```
self.calibrated = CalibratedClassifierCV(
    self.pipeline,
    method="isotonic",
    cv="prefit",
)
self.calibrated.fit(X_calib, y_calib)
```

3. Rebuild affected services:

```
docker compose up -d --build api scorer
```

After the fix, the API started successfully and the workers loaded the model.

Error 5: Bootstrap module not found

Observed error:

```
ModuleNotFoundError: No module named 'scripts'
```

Cause: `scripts/bootstrap.py` existed on the Windows host but `Dockerfile.api` did not copy the `scripts/` folder into the API container.

Fix: Add this line to `Dockerfile.api`:

```
COPY scripts/ ./scripts/
```

The relevant `Dockerfile` block became:

```
COPY app/ ./app/  
COPY migrations/ ./migrations/  
COPY alembic.ini .  
COPY scripts/ ./scripts/
```

Then rebuild API:

```
docker compose up -d --build api
```

Verify inside the container:

```
docker compose exec api ls -la /app/scripts
```

Then bootstrap:

```
docker compose exec api python -m scripts.bootstrap
```

Error 6: Scorer reported missing model file before bootstrap

Observed error:

```
FileNotFoundError: /app/artifacts/model_current.pkl
```

Cause: The scorer starts before the first model exists. The artifact is generated only when bootstrap/training runs.

Fix: This is expected on first startup. Run:

```
docker compose exec api python -m scripts.bootstrap
```

Then restart model-dependent services:

```
docker compose restart api scorer
```

Important: Old errors remain in `docker compose logs --tail=...`; use recent logs to judge current health:

```
docker compose logs --since=2m scorer
```

The successful worker state looked like:

```
Loading model artifact from /app/artifacts/model_current.pkl
Scorer worker started, consuming from Kafka topic: returns.incoming
Joined group 'sentinel-scorer'
Successfully synced group sentinel-scorer
```

Error 7: Kafka topic initialization warnings

Observed messages:

```
Topic returns.incoming is not available during auto-create initialization
Topic returns.incoming not found in cluster metadata
```

Cause: Kafka was still initializing and auto-creating the topic when the workers first subscribed.

Resolution: After Kafka startup and consumer-group rebalance, workers successfully joined and received a partition. This did not block final functionality.

Error 8: Swagger 403 Not authenticated

Observed response:

```
{
  "detail": "Not authenticated"
}
```

Cause: `/returns/score` is a protected endpoint and Swagger did not include a JWT Authorization header.

Fix:

1. Call `POST /auth/token` with raw ops/admin API key.
2. Copy the returned long `access_token`.
3. Click Swagger's **Authorize** button.
4. Paste the JWT in the format Swagger expects.

Important discovery: the Swagger `'HTTPBearer'` UI automatically adds `'Bearer'`, so pas

Error 9: Swagger Invalid or expired token

Cause: Old/invalid token persisted in Swagger session, or Bearer prefix was duplicated.

Fix:

- Click Swagger **Authorize**.
- Click **Logout** first if an old session exists.
- Get a fresh JWT from POST `/auth/token`.
- Paste only the token value if the UI prepends the scheme automatically.

Error 10: GET `/returns` returned empty list while database had the case

Observed:

```
{
  "items": [],
  "total": 0,
  "page": 1,
  "page_size": 50
}
```

But PostgreSQL confirmed ORD900001 existed and was scored Red.

Cause: Case-sensitive database filter values. The stored DB value was Red, but entering lowercase red in Swagger produced `band=red`. PostgreSQL compared strings case-sensitively:

```
red != Red
```

Fix: Enter the exact capitalized band:

```
Red
Yellow
Green
```

Or leave the band query field blank to retrieve all returns.

Future code improvement: Change API filter to case-insensitive matching, such as:

```
q = q.filter(ReturnCase.band.ilike(band))
```


Error 11: Dashboard did not show data after refresh

Observed behavior: A case appeared after using Incognito/private mode but disappeared or did not load in the regular browser session after refresh.

Cause: The React dashboard saves the JWT in browser local storage as:

```
sentinel_token
```

A refresh preserves local storage. The regular session likely retained a stale, malformed, or expired token, while Incognito started with empty storage and forced a fresh login.

Workaround used:

1. Open private/incognito mode with `Ctrl + Shift + N`.
2. Visit `http://localhost:3000`.
3. Log in with the **raw ops/admin API key**, not a JWT.
4. The frontend exchanges it for a new JWT automatically.

Permanent frontend improvement needed: Add an Axios response interceptor in `frontend/src/api/client.ts` immediately after the request interceptor:

```
apiClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401 || error.response?.status === 403) {
      clearToken();
      window.location.reload();
    }
    return Promise.reject(error);
  }
);
```

Then rebuild frontend:

```
docker compose up -d --build frontend
```

This makes the dashboard automatically clear an invalid session and display the login screen rather than silently showing an empty queue.

Error 12: Docker internal hostname versus host-browser URL confusion

Docker AI suggested changing frontend environment variables from `localhost` to `api`.

Why this was risky: The React application code executes in the user's Windows browser, not inside the frontend container. The Windows browser cannot resolve Docker's internal DNS hostname `api`.

Correct browser-facing frontend configuration:

```
environment:
  VITE_API_BASE_URL: http://localhost:8000
  VITE_WS_URL: ws://localhost:8000/ws/returns
```

If the browser console shows requests to `http://api:8000`, restore the localhost values above and rebuild:

```
docker compose up -d --build frontend
```

11. Current verified state

At the point this memory file was created, the following had been verified:

- Docker Desktop and WSL2 work.
- PostgreSQL started and accepts connections.
- Redis started and accepts connections.
- Kafka started; scorer workers successfully joined Kafka consumer group `sentinel-scorer`.
- FastAPI responded successfully at `/health`.
- Health response confirmed:

```
{
  "status": "ok",
  "model_loaded": true
}
```

- JWT/API-key authentication worked after correct Swagger authorization handling.
- A live request `ORD900001` was submitted successfully.
- Async Kafka scoring worked.
- Model classified it as Red with score 92.8.
- PostgreSQL persistence was confirmed manually.
- The dashboard was able to show it after a fresh browser login/session.

12. Recommended immediate next steps

A. Fix dashboard refresh/session UX

Implement the Axios 401/403 interceptor listed in Error 11, rebuild frontend, and test:

1. Log in normally at `http://localhost:3000`.
2. View Red case.
3. Refresh page.

4. Confirm it persists.

5. If token expires, confirm dashboard redirects to login instead of showing blank data.

B. Make API band filtering case-insensitive

Modify GET /returns in app/api/main.py:

```
if band:
    normalized_band = band.strip().capitalize()
    if normalized_band not in {"Green", "Yellow", "Red"}:
        raise HTTPException(status_code=422, detail="band must be Green, Yellow, or Red")
    q = q.filter(ReturnCase.band == normalized_band)
```

This prevents the confusing red versus Red behavior.

C. Create test cases for Green and Yellow

Use new IDs each time because Redis deduplicates recently submitted order IDs.

- Low-risk inputs should create a Green case.
- Moderate/ambiguous inputs should create a Yellow case.
- Strong stacked risk signals should create Red case.

D. Label outcomes

Use the dashboard evidence drawer to label cases as legitimate or confirmed abuse. Create at least 20 labeled cases to unlock meaningful live metrics and around 30+ to make the threshold simulator useful.

E. Demo rehearsal

Prepare three tabs/windows:

- Dashboard: <http://localhost:3000>
- Swagger: <http://localhost:8000/docs>
- Terminal in project directory: C:\Projects\return-abuse-sentinel

Useful emergency commands:

```
docker compose ps
docker compose logs --since=5m scorer
docker compose logs --tail=100 api scorer frontend
curl http://localhost:8000/health
```

13. Suggested buildathon narration

“Return-Abuse Sentinel protects merchants from post-purchase loss such as serial returns, refund-to-keep behavior, fake damage claims, item swaps, and coordinated return abuse. A secured FastAPI endpoint validates each return request, stores it in Postgres, and routes it via Kafka to independent scorer workers. The workers use a calibrated LightGBM risk model and SHAP explanations to produce a Green, Yellow, or Red decision band.

Green cases move through the normal return flow. Yellow cases receive low-cost verification. Red cases never receive an automated denial; instead, the platform prepares an evidence pack for a human reviewer. This makes the solution strictly defense-only.

We optimize thresholds for merchant-specific ₹ loss, including false-positive friction and manual-review costs, rather than reporting only accuracy. Reviewer outcomes feed precision/recall tracking, drift monitoring, and a champion-challenger retraining pipeline.”

14. Important safety reminders

- Never paste raw API keys or JWTs into chats, public repositories, screenshots, or presentations.
- Treat any JWT previously pasted publicly as compromised; it is short-lived, but rotate API keys/JWT secret for real deployment.
- The synthetic dataset is suitable for a demo/buildathon. It is not evidence that the system is production-ready on real merchant data.
- Real deployment requires privacy review, data minimization, consent/policy review, secure PII handling, proper label quality checks, and bias/fairness testing before customer-impacting use.
- Keep Red-band action human-in-the-loop. Never convert this demo into automatic refund denial or customer blocking without governance, monitoring, and merchant approval.